Björkdalen National Park

A website for hikers: trails, weather and a community
Programming & Methods

Ellinor Sundelin

ACE - Berghs 26/27

What is a database?

A place where a website saves information

Without one, everything disappears when you reload

Works like a spreadsheet: tables, columns and rows

Today: where it is · how the site talks to it · how to change it

Where is it?

A PostgreSQL database, hosted by Neon

In the cloud: Amazon's servers in Frankfurt

Close to Sweden, so it's fast

Free plan

How do I connect?

The key = a connection string (address + password)

Kept in .env, never on GitHub

Only the server uses it, never the browser

src/lib/server/db.js

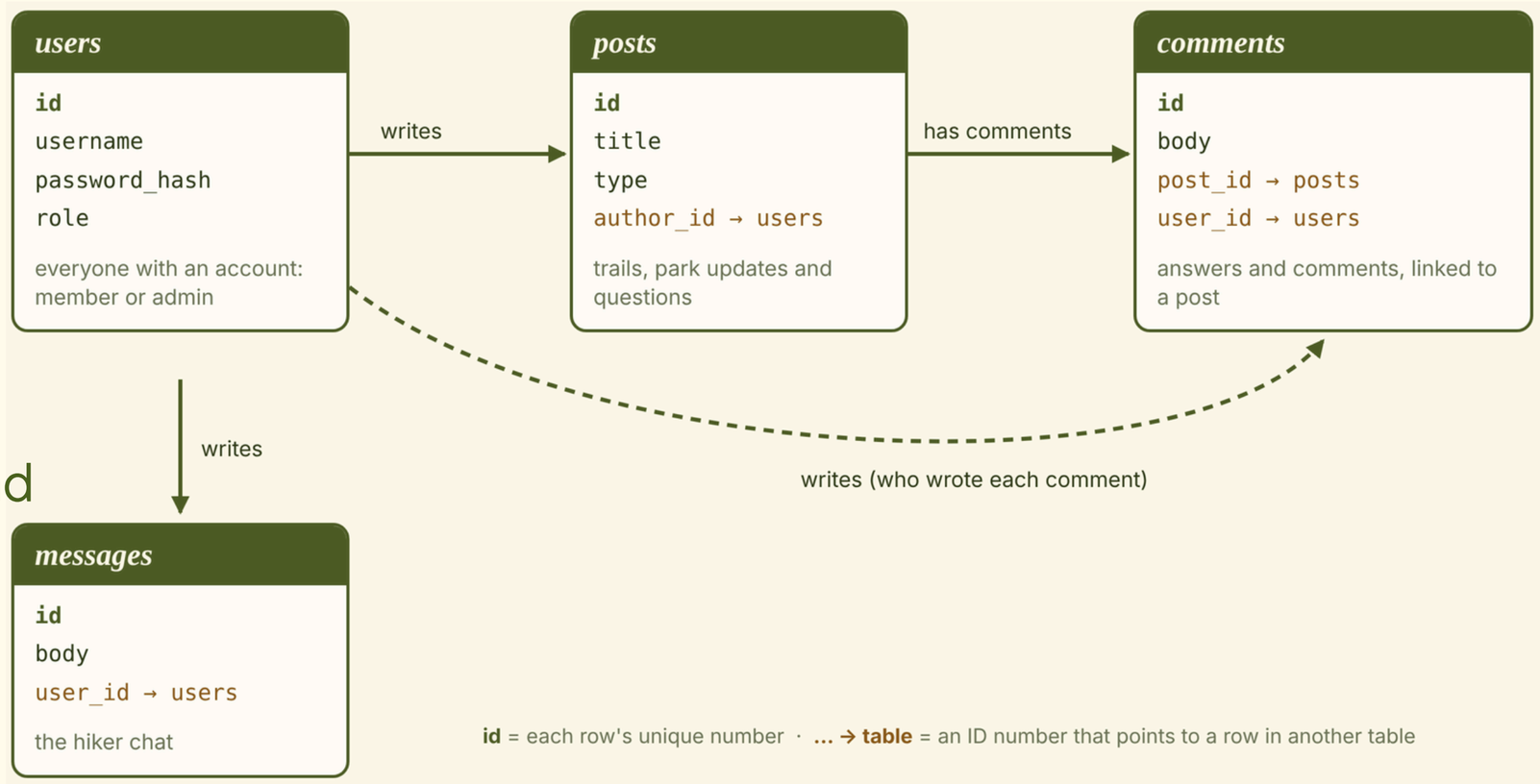
```
1  import { DATABASE_URL } from '$app/env/private';
2  import { neon } from '@neondatabase/serverless';
3
4  let database;
5
6  // Create the Neon client only when a server load function needs it.
7  export function getDatabase() {
8    if (!DATABASE_URL) {
9      throw new Error('DATABASE_URL is missing.');
10   }
11
12   if (!database) {
13     database = neon(DATABASE_URL);
14   }
15
16   return database;
17 }
```

My tables

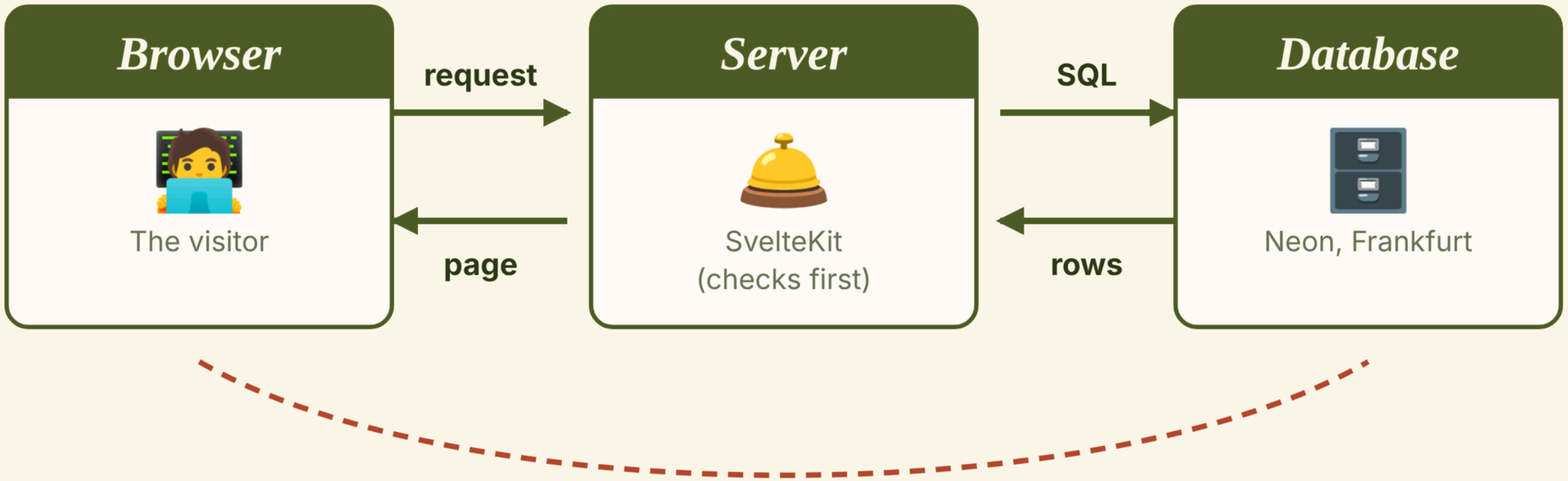
4 tables: users, posts, comments, messages

Linked by ID numbers

Each comment knows its post and who wrote it



How data flows



The browser never talks to the database. **The server is always in between.**

Reading data

1. The browser asks for the Trails page
2. The server runs load()
3. SELECT: get all trails from the database
4. The trails come back and fill the page

Writing data

1. Not signed in? No form, just "Sign in"
2. A signed-in hiker clicks Send
3. The server checks again: signed in?
4. INSERT: save the message
5. The page reloads with the new message

src/routes/community/+page.server.js · sendMessage

```
sendMessage: async ({ request, locals }) => {
  // 1. The waiter checks: is the user signed in?
  if (!locals.user) {
    return fail(401, { error: 'Sign in to take part.' });
  }

  // 2. Read the message from the form
  const formData = await request.formData();
  const body = String(formData.get('body') ?? '').trim();

  // 3. Ask the storeroom to save it
  const sql = getDatabase();
  await sql`
    INSERT INTO messages (user_id, body)
    VALUES (${locals.user.id}, ${body})
  `;

  // 4. Reload the page, showing the chat
  redirect(303, '/community#chat');
}
```

Changing what's stored

Changing the data (on the site)
INSERT adds · UPDATE changes · DELETE removes

Changing the structure (by hand in Neon)
CREATE TABLE · ALTER TABLE

Safety: user text is sent separately from the SQL, so it can't run as code

My process & problems

Copilot wrote the code, one feature at a time

Prompts: one feature each · "explain this code" · pasted errors

Docs: Neon and SvelteKit

Problem: a 500 error – a column didn't exist yet

Lesson: change the database first, then the code

Questions?